NAME OF PROJECT:

- Uno: Java Edition

GROUP MEMBERS:

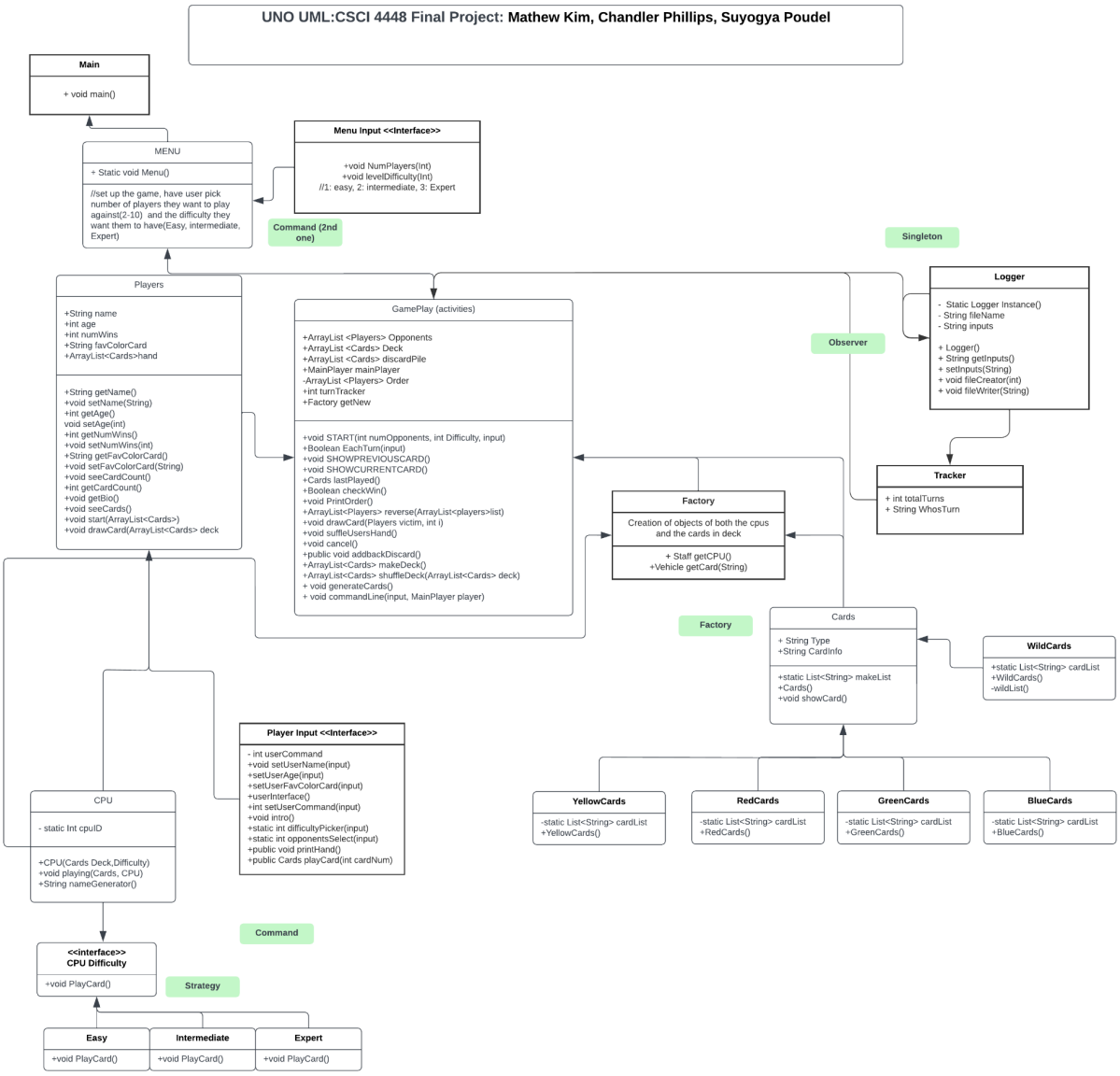
- Mathew Kim
- Chandler Phillips
- Suyogya Poudel

FINAL STATE OF SYSTEM STATEMENT:

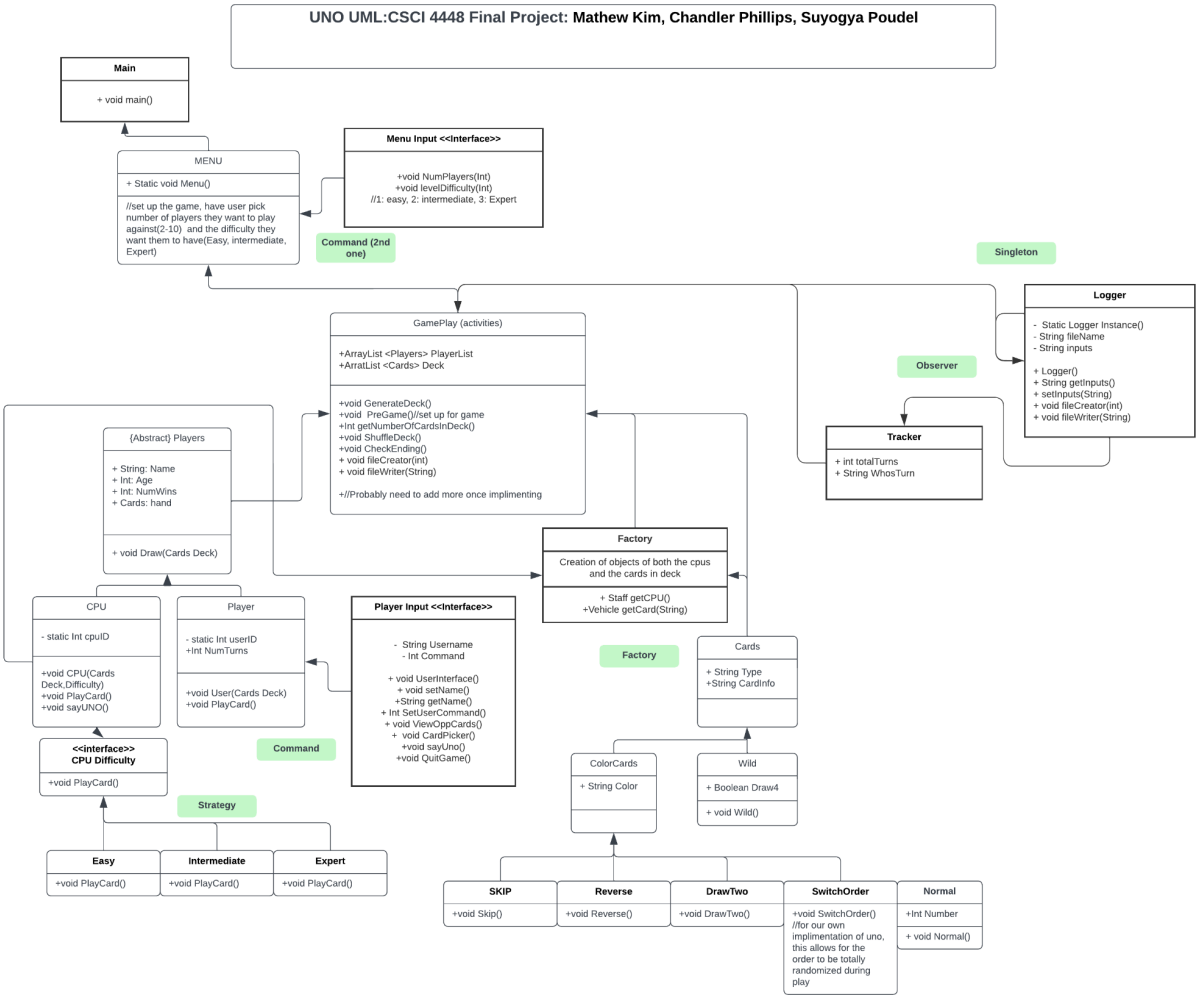
- The final state of our system is finished, with the cards and user commands in the Uno command line game all functioning correctly. While the base game is fully functional and works as intended, severe polishing is recommended in case of future iterations. Having the game run on the command line, essentially a game as intensive as Uno, is never a good idea, and should ultimately be moved to either a game engine or a web application where full User interactability can be conducted. Having the game run on a command line is messy, unintuitive, and can cause massive strain on potential users' eyes.
- In terms of things that were changed, not much was altered as the original scope of recreating Uno in the command line utilizing Java in an Object Oriented fashion still largely remained consistent.
- If quite possibly more time was given, or the amount of student workload was lessened, the project could have quite possibly expanded, but with the way the game is now, "Uno: Java Edition," is fully functional and ultimately achieves the goal we set out to do

FINAL CLASS DIAGRAM AND COMPARISON STATEMENT:

- **Project 7:**


UNO UML:CSCI 4448 Final Project: Mathew Kim, Chandler Phillips, Suyogya Poudel
Main
+ void main()
MENU
+ Static void Menu()
//set up the game, have user pick number of players they want to play against(2-10) and the difficulty they want them to have(Easy, intermediate, Expert)
Menu Input <<Interface>>
+void NumPlayers(Int)
+void levelDifficulty(Int)
//1: easy, 2: intermediate, 3: Expert
Command (2nd one)
Singleton
Logger
- Static Logger Instance()
- String fileName
- String inputs
+ Logger()
+ String getInputs()
+ setInputs(String)
+ void fileCreator(int)
+ void fileWriter(String)
Observer
Players
+String name
+int age
+int numWins
+String favColorCard
+ArrayList<Cards>hand
+String getName()
+void setName(String)
+int getAge()
void setAge(int)
+int getNumWins()
+void setNumWins(int)
+String getFavColorCard()
+void setFavColorCard(String)
+void seeCardCount()
+int getCardCount()
+void getBio()
+void seeCards()
+void start(ArrayList<Cards>)
+void drawCard(ArrayList<Cards> deck
GamePlay (activities)
+ArrayList <Players> Opponents
+ArrayList <Cards> Deck
+ArrayList <Cards> discardPile
+MainPlayer mainPlayer
-ArrayList <Players> Order
+int turnTracker
+Factory getNew
+void START(int numOpponents, int Difficulty, input)
+Boolean EachTurn(input)
+void SHOWPREVIOUSCARD()
+void SHOWCURRENTCARD()
+Cards lastPlayed()
+Boolean checkWin()
+void PrintOrder()
+ArrayList<Players> reverse(ArrayList<players>list)
+void drawCard(Players victim, int i)
+void suffleUsersHand()
+void cancel()
+public void addbackDiscard()
+ArrayList<Cards> makeDeck()
+ArrayList<Cards> shuffleDeck(ArrayList<Cards> deck)
+ void generateCards()
+ void commandLine(input, MainPlayer player)
Tracker
+ int totalTurns
+ String WhosTurn
Factory
Creation of objects of both the cpus and the cards in deck
+ Staff getCPU()
+Vehicle getCard(String)
Factory
Cards
+ String Type
+String CardInfo
+static List<String> makeList
+Cards()
+void showCard()
WildCards
+static List<String> cardList
+WildCards()
-wildList()
Player Input <<Interface>>
- int userCommand
+void setUserName(input)
+setUserAge(input)
+setUserFavColorCard(input)
+userInterface()
+int setUserCommand(input)
+void intro()
+static int difficultyPicker(input)
+static int opponentsSelect(input)
+public void printHand()
+public Cards playCard(int cardNum)
YellowCards
-static List<String> cardList
+YellowCards()
RedCards
-static List<String> cardList
+RedCards()
GreenCards
-static List<String> cardList
+GreenCards()
BlueCards
-static List<String> cardList
+BlueCards()
CPU
- static Int cpuID
+CPU(Cards Deck,Difficulty)
+void playing(Cards, CPU)
+String nameGenerator()
Command
<<interface>>
CPU Difficulty
+void PlayCard()
Strategy
Easy
+void PlayCard()
Intermediate
+void PlayCard()
Expert
+void PlayCard()

- **Project 5:**



- **Key Changes:**
    - Overall, not much has been altered from the UML diagram shown in Project 5. If anything, more has been added such as the presence of more functions in order to consolidate core functionality without having the need to repeat code, as well as an “AllPlayers” class, that describes actions taken in which the order of players and cards are to be affected

THIRD-PARTY CODE VS ORIGINAL CODE STATEMENT:

- Most of the code was directly inspired by the FNCD as well as our general knowledge of Uno. It is quite painful and clear to say that the originality of our work is quite evident, considering that the codebase is pretty messy given the amount of time and work we were given. However, various debugging was helped due to services such as StackOverflow, and Baeldung, where most tips about patterns and general Java tips were given

STATEMENT ON OOAD PROCESS FOR SEMESTER PROJECT:

- Time management was definitely an issue with this current project. ⅔ of our group consisted of seniors who needed to get their capstone done, as well as taking into consideration various assignments and schoolwork for all of us.
- Code management was also an issue, where things we planned for OFTEN did not proceed the way we wanted them to. For instance, the numerous added functions were added purely to avoid clutter, but the group ended up making it anyways. Further proceedings must be taken with caution and careful planning.
- A lot of code was inspired by our FNCD assignment and we took numerous design elements and often went back to it for reference.